

ENGINEERING SPECIFICATION – META MULTI-CHANNEL SAAS

ReelToRealAuto Master Plan

Facebook, Instagram aur WhatsApp ko ek multi-tenant automation SaaS me jodne ka complete blueprint — tenancy model, connection flows, token lifecycle, webhook spine, domain schema, policy windows, compliance aur build roadmap.

TARGET GRAPH API	CHANNELS	TENANCY	BLOCKING PATH
v24.0 (env-configurable)	FB Page · IG Pro · WABA	Org → Connection → Channel	App Review + Biz Verification

CONTENTS

01	Scope & Non-Goals	02	Meta Platform Reality
03	Tenancy & Identity Model	04	Layer 1 — App Login
05	Layer 2 — Channel Connect	06	Permissions & App Review
07	Token Lifecycle & Health	08	Webhook Spine
09	Domain Model & Schema	10	Messaging Policy Windows
11	Outbound Pipeline	12	Reels Publishing
13	Automation & AI Engine	14	Security & Secrets
15	Observability	16	Environments & Config
17	Billing & Metering	18	Build Roadmap
19	Open Decisions		

ReelToRealAuto ek **multi-tenant SaaS** hai jisme ek business apne Facebook Page, Instagram Professional account aur WhatsApp Business number ko connect karke inbound conversations (DM, comment, WA message) par automated + AI-driven replies chala sake, aur outbound content (Reels, posts, WA campaigns) schedule kar sake.

V1 ME KYA HAI

- Teeno channels ka independent connect / disconnect
- Unified inbox: Contact → Conversation → Message (platform-agnostic)
- Comment automation (keyword → public reply + private DM)
- AI auto-reply knowledge base ke basis par, human handoff ke saath
- Instagram + Facebook Reels publishing & scheduling
- WhatsApp template messaging with opt-in ledger

V1 ME KYA NAHI (EXPLICIT NON-GOALS)

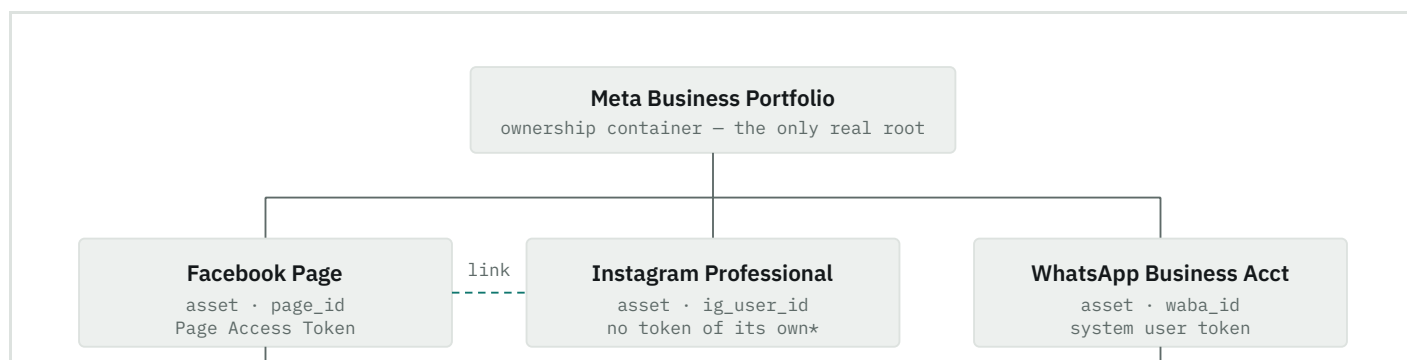
- Ads / Marketing API (alag review track, alag permissions — abhi nahi)
- Threads, Messenger for non-Page, Facebook Groups
- Payments in-chat (WhatsApp Pay / Meta Pay) — Phase 2+
- White-label reseller / sub-agency hierarchy (schema me room chhoda hai, feature nahi)

DESIGN PRINCIPLE

Har channel **independently degrade** hona chahiye. Ek channel ka token marne se doosre channel ka inbox, automation ya publishing nahi rukna chahiye. Ye poore schema aur job design ko drive karta hai.

02 Meta Platform Reality

Meta ke backend me do alag duniya hain. **Accounts Center** (personal FB profile + personal IG, cross-posting ke liye) — SaaS ke liye iska koi use nahi. Aur **Business Portfolio** (purana naam: Business Manager) — yahi asli backbone hai jahan teeno assets rehte hain.





Paanch cheezein jo log galat samajhte hain

CORRECTION 01 — "PAGE TOKEN NEVER EXPIRES"

Token expiry aur **Data Access Expiration** do alag mechanisms hain. Long-lived user token से बना Page token का कोई `expires_at` नहीं होता, लेकिन अगर authorizing user ~90 दिन तक app के साथ re-authorize/interact ना करे तो *data access* expire हो जाता है और calls fail होने लगती हैं। इसके अलावा token मरता है: password change, user का app remove करना, page role हटाना, business verification lapse, या Meta का policy action. Matlab "never expires" पर architecture मत बनाओ — **health check + reconnect flow mandatory है** (§7).

CORRECTION 02 — "EK AUTHORIZATION ME TEENO CONNECT HO JAAYENGE"

ऐसा नहीं होता. FB + IG एक ही Facebook Login for Business dialog में आ सकते हैं। **WhatsApp Embedded Signup** हमेशा **alag dialog + alag config_id** है। UI में "Connect everything" बटन दिखाना ठीक है, पर backend में दो sequential flows challenge.

CORRECTION 03 — CLASSIC FB.LOGIN

Raw `scope` string वाले classic Facebook Login की जगह **Facebook Login for Business** (`config_id` based) use करो. Asset selection Meta के अपने dialog में होता है, granular permission grant मिलता है, और App Review इस assumption पर होता है।

CORRECTION 04 — GRAPH VERSION PIN

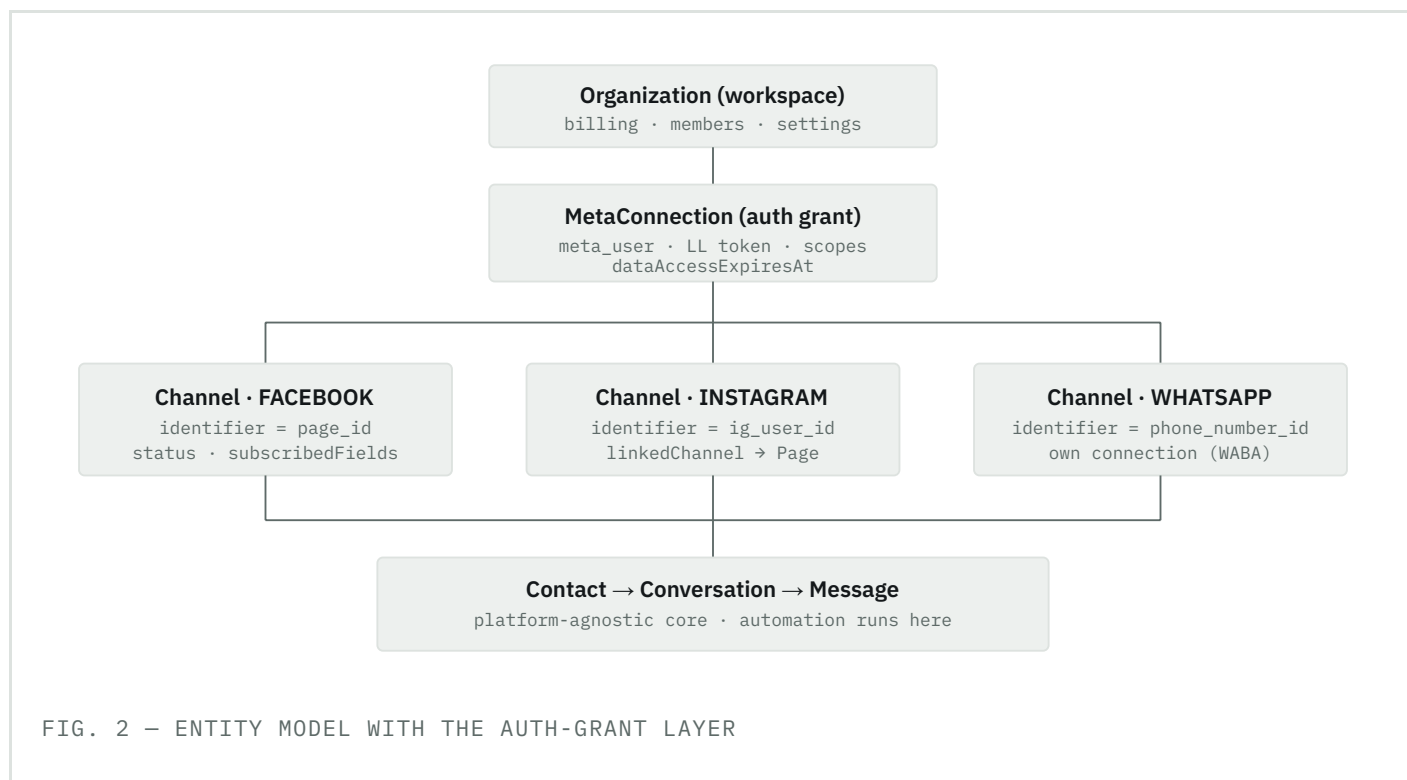
v21.0 Oct 2024 का है; current v24.0 (Oct 2025) है और Meta के versions roughly 2 साल में sunset होते हैं। Version को **env var** बनाओ, code में hardcoded मत करो, और हर नये version पर एक upgrade ticket (§16).

CORRECTION 05 — STANDALONE IG KE SCOPE NAMES ALAG HAIN

Instagram API with Instagram Login में permissions `instagram_business_basic`, `instagram_business_manage_messages`, `instagram_business_manage_comments`, `instagram_business_content_publish` हैं — Page-linked flow के `instagram_*` scopes से अलग namespace. Dono flows support karne का matlab है **do alag token strategies, do alag refresh policies, do alag endpoints** — support cost realistically socho

Tenancy & Identity Model

Root entity hamesha **Organization** hai, Meta nahi. Lekin Organization aur Channel ke beech ek layer chahiye jo research plan me missing thi: **MetaConnection** — yaani "kis Meta user ne, kaunse scopes ke saath, kab authorize kiya".



MetaConnection kyun zaroori hai

Page Access Token us user se derive hota hai jisne authorize kiya. Wo banda company chhod de, password badle, ya app remove kar de — uske saare channels ek saath mar jaate hain. Agar grant alag entity hai to:

- Ek re-auth se saare dependent channels ek saath refresh ho jaate hain
- Aapko pata hota hai kaunsa channel kis grant par depend karta hai (blast radius visible)
- Granted scopes store hain — feature gating possible ("Reels publish karne ke liye extra permission chahiye, reconnect karein")
- `dataAccessExpiresAt` track karke **expire hone se pehle** user ko nudge kar sakte ho

Ye baad me retrofit karna bahut dardnaak hai — Day 1 par daal do.

Original schema me `@unique([orgId, platform, channelIdIdentifier])` tha. Iska matlab do alag orgs same Facebook Page connect kar sakte hain. Webhook me sirf `page_id` aata hai — ab kaunse org ka event hai? Ambiguous. Active channels par **global unique [platform, channelIdIdentifier]** rakho, aur duplicate connect attempt par explicit "ye asset already kisi aur workspace se connected hai" error + takeover request flow do.

Roles & membership

Original plan me Organization ka koi owner hi nahi tha. Minimum: `User`, `OrgMember` (role: `OWNER` | `ADMIN` | `AGENT` | `VIEWER`), `Invite`. Role matter karta hai kyunki channel connect/disconnect aur billing sirf `OWNER/ADMIN` kar sakein, jabki `AGENT` sirf inbox dekhe.

04 Layer 1 — App Login

App authentication aur social authorization ko **kabhi mix mat karo**. Ye plan ka sabse sahi decision tha — isko strictly enforce karo.

- Login options: Email + password (argon2id), Magic link, Google OAuth. Facebook Login ko identity provider ke roop me *optional* rakho — agar user ka FB account disable hua to wo apne SaaS account se hi lock out ho jaayega.
- Session: short-lived access JWT (15 min) + rotating refresh token (httpOnly cookie), device/session list with revoke.
- Org switching: JWT me `activeOrgId`, har request par membership verify — **kabhi client se aaya orgId trust mat karo**.
- 2FA (TOTP) at least `OWNER` role ke liye — kyunki ek compromised login ka matlab client ke saare Meta assets tak access hai.

MULTI-TENANCY GUARDRAIL

Har query me `orgId` scope Prisma middleware / extension se inject karo, developer discipline par mat chhodo. Ek cross-tenant leak SaaS ko khatam kar deta hai.

05 Layer 2 — Channel Connect Flows

Dashboard → Settings → Channels par teen alag buttons. Koi bhi flow doosre par depend nahi karta.

5.1 • FACEBOOK • INSTAGRAM • LOGIN FOR BUSINESS

- 01 Frontend `FB.login()` with `config_id` (Login for Business configuration),
`response_type: 'code'`, `override_default_response_type: true`.
- 02 Backend code → **long-lived user token** exchange (`grant_type=fb_exchange_token`),
aur `GET /debug_token` se `scopes`, `expires_at`, `data_access_expires_at` nikaal
kar `MetaConnection` me save.
- 03 Asset discovery:

```
GET /v24.0/me/accounts
    ?fields=id,name,username,picture,access_token,
      instagram_business_account{id,username,name,profile_picture_url}
```

- 04 User ko selection modal — kaunse Pages / linked IG accounts connect karne hain. (Agar
50+ pages hain to pagination handle karo.)
- 05 Confirm par har selected asset ke liye `Channel row`, token encrypt karke store.
- 06 **Webhook subscribe — ye step research plan me missing tha aur iske bina kuch nahi
aayega:**

```
POST /v24.0/{page-id}/subscribed_apps
    ?subscribed_fields=messages,messaging_postbacks,messaging_optins,
      message_reactions,message_deliveries,message_reads,feed,mention
    &access_token={PAGE_TOKEN}
```

Instagram ke liye linked Page ke through same call se IG fields bhi subscribe hote hain;
standalone IG login me `POST /{ig-id}/subscribed_apps`.

- 07 Verification: `GET /{page-id}/subscribed_apps` se confirm karo ki fields actually
subscribe hue, aur ek test event log karo. Channel tab `ACTIVE` mark karo — token save
hote hi nahi.

5.2 • INSTAGRAM standalone (Business Login)

Un creators ke liye jinke paas Facebook Page nahi hai. Alag OAuth endpoint, alag scope
namespace, aur **60-day token** jise refresh karna padta hai.

- Refresh: `GET /refresh_access_token?grant_type=ig_refresh_token` — token kam se
kam **24 ghante purana** hona chahiye aur **expire hone se pehle**. Expire ho gaya to sirf full
re-login.
- Cron: daily job jo `tokenExpiresAt < now + 10 days` wale IG channels refresh kare,
failure par `FFDS REAUTH`.

Facebook par NEEDS_REQUIRED :

- Limitation UI me clearly likho: is mode me Facebook Page features aur cross-posting available nahi.

5.3 • WHATSAPP — Embedded Signup

- 01 FB.login() with WhatsApp-specific config_id + extras.setup (prefill business name/email se friction kam hota hai).
- 02 Meta ka modal: business select/create → phone number → OTP verify → WABA create.
- 03 SDK message event se waba_id + phone_number_id , aur callback se code .
- 04 Backend: code → **business access token** exchange (/oauth/access_token with app credentials).
- 05 POST /v24.0/{waba-id}/subscribed_apps — WABA ko aapke app ke webhooks se subscribe karo.
- 06 **Phone number register — bina iske ek bhi message send nahi hoga:**

```
POST /v24.0/{phone-number-id}/register
{ "messaging_product": "whatsapp", "pin": "<6-digit two-step PIN>" }
```

PIN ko securely store karo — number re-register / migration ke time chahiye hoga.

- 07 Health read: GET /{phone-number-id}?
fields=verified_name,quality_rating,code_verification_status,throughput → metadata me save.
- 08 Billing: WABA par payment method attach hona chahiye, warna messages fail honge. Decide karo — client ka apna method, ya aap Tech Provider ke roop me credit line share karoge (§17, §19).

5.4 • Disconnect flow (aksar bhool jaate hain)

- DELETE /{page-id}/subscribed_apps (ya WABA equivalent) — warna events aate rahenge aur aap unhe drop karte rahoge
- Token rows purge (soft-delete nahi — actual secret wipe), queued jobs cancel, scheduled posts cancel
- Meta se aane wale **deauthorize callback** aur **data deletion callback** par bhi yahi path chale (§6)
- Conversations/messages ka kya hoga — retain with tombstone, ya delete? §14 me policy fix karo.

Permissions & App Review

BIGGEST GAP IN THE ORIGINAL PLAN

Advanced Access ke bina aapka app sirf app ke **admins, developers aur testers** par chalega. Ek bhi real client onboard nahi hoga. App Review + Business Verification realistically **4–6 hafte** leta hai aur iterations hote hain — isliye ye Day 0 parallel track hai, launch se pehle ka kaam nahi.

Required permissions

PERMISSION	KIS LIYE	CHANNEL
pages_show_list	Page discovery in connect flow	FB
pages_read_engagement	Page content, comments read	FB
pages_manage_metadata	Webhook subscribe — iske bina events nahi aayenge	FB
pages_manage_posts	Post / Reel publish	FB
pages_manage_engagement	Comment reply, hide, delete	FB
pages_messaging	Messenger send / receive	FB
instagram_basic	IG account + media read	IG
instagram_manage_comments	Comment reply + private_replies	IG
instagram_manage_messages	IG DM	IG
instagram_content_publish	Reels / feed publish	IG
instagram_manage_insights	Analytics (optional V1)	IG
business_management	Business asset access	all
whatsapp_business_management	WABA, templates, phone numbers	WA
whatsapp_business_messaging	Send / receive messages	WA

Review submission checklist

- **Business Verification** — legal entity docs, registered address, phone/domain verification. Sabse pehle shuru karo, isme sabse zyada time lagta hai.
- **Privacy Policy URL + Terms of Service URL** — live, public, app ke actual data use ko describe karte hue
- **Data Deletion Request callback** — mandatory endpoint jo user ka data delete kare aur confirmation code + status URL return kare
- **Deauthorize callback** — user app remove kare to aapke paas signed request aata hai; channel ko DISCONNECTED karo
- **Per-permission screencast** — har permission ka real use dikhana hoga, step-by-step, aapke live UI me
- **Test credentials** — reviewer ke liye ek working account + test Page/IG/WABA
- **Tech Provider** status (WhatsApp ke liye) — agar aap clients ke behalf par WABA onboard kar rahe ho
- App icon, category, use-case description — reviewer ko 60 second me samajh aana chahiye ki app kya karta hai

PRACTICAL TIP

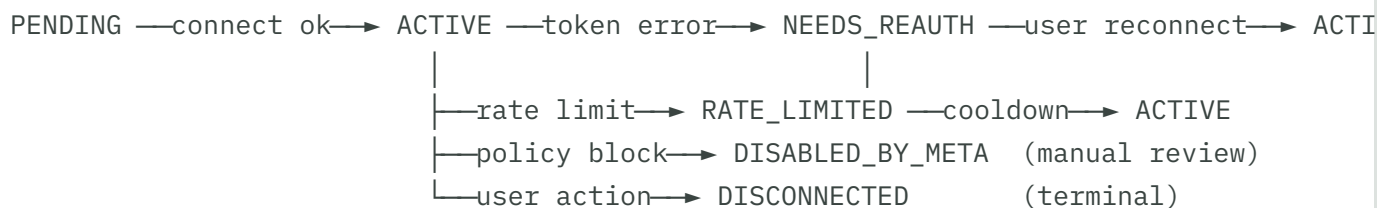
Review ke liye ek **demo org** hamesha ready rakho jisme sab kuch pre-connected ho. Reviewer agar onboarding me hi atak gaya to rejection aata hai, aur har rejection cycle 1–2 hafte ka hai.

07

Token Lifecycle & Channel Health

`isActive`: Boolean kaafi nahi hai. SaaS me sabse zyada support tickets yahin se aate hain — "mera automation band ho gaya" ka 80% cause dead token hota hai. Isliye proper state machine chahiye.

CHANNEL STATUS MACHINE



META ERROR CODE → ACTION MAPPING

CODE	MEANING	ACTION
------	---------	--------

190	Access token invalid	→ NEEDS_REAUTH
190 / 458	User ne app remove kar diya	→ NEEDS_REAUTH, reconnect email
190 / 460	Password change	→ NEEDS_REAUTH
190 / 463	Token expired / data access expired	→ NEEDS_REAUTH
190 / 467	Session invalidated	→ NEEDS_REAUTH
190 / 459	User checkpointed	Notify user, retry later
200 / 10 / 3	Permission missing	Re-auth with upgraded scopes
4 / 17 / 32 / 613	Rate limit	→ RATE_LIMITED + exponential backoff
368	Temporarily blocked — policy	→ DISABLED_BY_META, alert on-call
100	Invalid parameter	Bug — alert, do not retry
131047	WA: 24h window closed	Template par switch karo
131026	WA: message undeliverable	Contact flag, stop retry
132xxx	WA: template error	Template status refresh

HEALTH CRON (DAILY)

- Har active channel par GET /debug_token → is_valid , scopes , data_access_expires_at
- data_access_expires_at 14 din ke andar hai → proactive "reconnect" email + in-app banner
- Granted scopes se koi required scope hat gaya → feature disable + prompt
- IG standalone tokens jo 10 din me expire honge → auto-refresh attempt
- WA: quality_rating = YELLOW / RED → org ko alert, marketing sends throttle karo

RECONNECT UX

Dashboard par persistent banner: "Instagram @handle disconnected — Reconnect". Email + in-app notification. Sabse important: automation rules ko **delete mat karo**, sirf pause karo, taaki reconnect par sab wapas chalu ho jaye.

Ek hi endpoint teeno platforms ke events receive karta hai — ye plan me sahi tha. Lekin handler me payload ko seedha process karna production me teen tarike se toot-ta hai: security, latency aur duplicates.

NON-NEGOTIABLE RULES

- 01
- GET verification** — `hub.mode=subscribe` , `hub.verify_token` match, `hub.challenge` plain text return.
- 02
- Signature verification** — `X-Hub-Signature-256` = `HMAC-SHA256(raw body, app_secret)`. P0 Iske bina koi bhi aapke endpoint par fake events bhej sakta hai — ye ek open security hole hai. Raw body chahiye, isliye JSON parse se pehle `rawBody` capture karo (`NestJS` me `bodyParser.json({ verify })`).
- 03
- <5 second me 200** — warna Meta retry karega aur repeated failure par webhook subscription disable kar dega. Pattern: *verify → raw event persist → enqueue → 200*. Saara processing worker me.
- 04
- Idempotency** — duplicate delivery normal hai. Platform message id / comment id par unique constraint, aur processing se pehle dedupe check.
- 05
- Out-of-order** — ordering guaranteed nahi hai. Message ordering timestamp se karo, arrival order se nahi.
- 06
- Batching** — ek payload me multiple `entry` , aur ek entry me multiple `changes` / `messaging` items aate hain. Har item alag job.

ROUTING TABLE

OBJECT	IDENTIFIER PATH	CHANNEL LOOKUP
page	<code>entry[].id</code>	FACEBOOK + <code>page_id</code>
instagram	<code>entry[].id</code>	INSTAGRAM + <code>ig_user_id</code>
whatsapp_business_account	<code>changes[].value.metadata.phone_number_id</code>	WHATSAPP + <code>phone_number_id</code>

HANDLER SHAPE

```
WEBHOOK.CONTROLLER.TS

@Post('webhook/meta')
```

```

@HttpCode(200)
async handle(@Req() req: RawBodyRequest<Request>, @Body() payload: MetaWebhookDto) {
  // 1. verify – reject unsigned traffic before touching the payload
  if (!this.crypto.verifySignature(req.rawBody, req.headers['x-hub-signature-256']))
    throw new ForbiddenException();
  }

  // 2. persist raw (replay ke liye) – ye table baad me lifesaver hai
  const event = await this.events.store({
    object: payload.object,
    payload,
    receivedAt: new Date(),
  });

  // 3. fan out to queue – koi business logic yahan nahi
  for (const entry of payload.entry ?? []) {
    await this.queue.add('meta-event', { eventId: event.id, object: payload.object, entry,
      { jobId: `${event.id}:${entry.id}:${entry.time}`, attempts: 5,
        backoff: { type: 'exponential', delay: 2000 } } });
  }

  return { received: true }; // 4. always fast 200
}

```

WORKER SIDE

- object se pipeline choose karo → channel resolve → orgId inject → domain event
- Channel nahi mila (disconnect ke baad late event) → log + drop, error mat throw karo
- Failure par retry with backoff, 5 attempts ke baad **DLQ** + alert
- **Replay tooling** — raw event table se kisi bhi time range ko re-process karne ka admin command. Jab bug fix karoge to ye chahiye hi chahiye.
- Per-org concurrency limit taaki ek bada client sabka queue block na kare

SUBSCRIBE FIELDS REFERENCE

CHANNEL	FIELDS
Page	messages, messaging_postbacks, messaging_optins, messaging_referrals, message_deliveries, message_reads, feed, mention
Instagram	messages, message_reactions, comments, live_comments, mentions, story_insights
WABA	messages, message_template_status_update, template_category_update, phone_number_quality_update, account_update, account_alerts

Domain Model & Schema

Original plan me sirf Channel tha. Product actually Contact → Conversation → Message ke bina chal hi nahi sakta. Neeche production-shape schema hai.

IDENTITY RESOLUTION

Ek hi insaan ka Messenger PSID, Instagram IGSID aur WhatsApp number teeno alag hote hain — aur PSID/IGSID **per-app scoped** hote hain, yaani globally unique nahi. Isliye Contact per-channel identity rakho, aur ek optional Person merge layer (phone/email match par) baad ke liye chhod do. V1 me over-engineer mat karo, par column space rakho.

SCHEMA.PRISMA — CORE

```
// _____ tenancy _____
model Organization {
  id          String      @id @default(uuid())
  name        String
  slug        String      @unique
  planId      String?
  timezone    String      @default("Asia/Kolkata")
  members     OrgMember[]
  connections MetaConnection[]
  channels     Channel[]
  contacts     Contact[]
  createdAt   DateTime    @default(now())
}

model User {
  id          String      @id @default(uuid())
  email       String      @unique
  passwordHash String?
  name        String?
  totpSecret  String?     // encrypted
  memberships OrgMember[]
}

model OrgMember {
  id          String @id @default(uuid())
  orgId       String
  userId      String
  role        Role   @default(AGENT)
  org         Organization @relation(fields: [orgId], references: [id], onDelete: Cascade)
  user        User      @relation(fields: [userId], references: [id], onDelete: Cascade)
```

```

    @@unique([orgId, userId])
}

enum Role { OWNER ADMIN AGENT VIEWER }

// _____ meta auth grant _____
model MetaConnection {
  id                String          @id @default(uuid())
  orgId             String
  org               Organization @relation(fields: [orgId], references: [id], onDelete: Cascade)

  provider          Provider        // FACEBOOK_BUSINESS | INSTAGRAM_LOGIN | WHATSAPP_ESU
  metaUserId        String?         // authorizing user (FB flows)
  businessId        String?         // Meta Business Portfolio id
  userTokenEncrypted String
  keyVersion         Int             @default(1)
  grantedScopes      String[]
  tokenExpiresAt     DateTime?
  dataAccessExpiresAt DateTime?     // ← 90-day trap, isse monitor karo
  status             ConnStatus @default(ACTIVE)
  lastCheckedAt      DateTime?

  channels           Channel[]
  createdAt          DateTime @default(now())
  @@unique([orgId, provider, metaUserId])
}

enum Provider { FACEBOOK_BUSINESS INSTAGRAM_LOGIN WHATSAPP_ESU }
enum ConnStatus { ACTIVE NEEDS_REAUTH REVOKED }

// _____ channels _____
model Channel {
  id                String          @id @default(uuid())
  orgId             String
  org               Organization @relation(fields: [orgId], references: [id], onDelete: Cascade)
  connectionId      String?
  connection         MetaConnection? @relation(fields: [connectionId], references: [id], onDelete: Cascade)

  platform           PlatformType
  channelIdIdentifier String          // page_id | ig_user_id | phone_number_id
  name               String
  handle             String?         // @username | +91XXXXXXXXXX
  avatarUrl          String?

  accessTokenEncrypted String
  keyVersion         Int             @default(1)
  tokenExpiresAt     DateTime?
}

```

```

status          ChannelStatus @default(PENDING)
lastError        Json?
lastHealthCheckAt DateTime?
subscribedFields String[]
graphVersion     String          @default("v24.0")

// self-relation: IG ↔ its parent Page
linkedChannelId  String?
linkedChannel    Channel?        @relation("ChannelLink", fields: [linkedChannelId, linkedFrom])
linkedFrom       Channel[]       @relation("ChannelLink")

metadata         Json?           // { wabaId, pageId, igUsername, qualityRating, ... }

conversations     Conversation[]
createdAt         DateTime       @default(now())
updatedAt         DateTime       @updatedAt

@@unique([platform, channelIdIdentifier]) // GLOBAL – webhook routing must be unambiguous
@@index([orgId, platform])
@@index([status])
}

enum PlatformType { FACEBOOK INSTAGRAM WHATSAPP }
enum ChannelStatus { PENDING ACTIVE NEEDS_REAUTH RATE_LIMITED DISABLED_BY_META DISCONNECTED }

```

SCHEMA.PRISMA – CONVERSATIONS

```

model Contact {
  id          String @id @default(uuid())
  orgId       String
  org         Organization @relation(fields: [orgId], references: [id], onDelete: Cascade)
  platform    PlatformType
  platformUserId String // PSID | IGSID | wa_id (E.164)
  name        String?
  username    String?
  phone       String?
  email       String?
  locale      String?
  personId    String? // future cross-channel merge
  optIns      OptIn[]
  conversations Conversation[]
  tags        String[]
  attributes  Json?
  @@unique([orgId, platform, platformUserId])
  @@index([orgId, phone])
}

```

```

model Conversation {
  id          String    @id @default(uuid())
  orgId       String
  channelId   String
  channel     Channel   @relation(fields: [channelId], references: [id], onDelete: CASCADE)
  contactId   String
  contact     Contact   @relation(fields: [contactId], references: [id], onDelete: CASCADE)

  lastInboundAt DateTime? // ← 24h window calculation ka source of truth
  lastOutboundAt DateTime?
  windowExpiresAt DateTime? // derived, indexed for cheap queries
  status       ConvStatus @default(OPEN)
  assignedTo   String?    // OrgMember id – AI se human handoff
  aiEnabled    Boolean    @default(true)
  unreadCount  Int        @default(0)

  messages     Message[]
  @@unique([channelId, contactId])
  @@index([orgId, status, lastInboundAt])
}

```

```

enum ConvStatus { OPEN SNOOZED CLOSED }

```

```

model Message {
  id          String    @id @default(uuid())
  orgId       String
  conversationId String
  conversation Conversation @relation(fields: [conversationId], references: [id])

  platformMessageId String? // idempotency key
  direction         Direction
  type              MsgType  @default(TEXT)
  body              String?
  mediaUrl          String?
  payload           Json?    // raw platform payload

  status          MsgStatus @default(PENDING)
  errorCode       String?
  sentBy          SentBy    @default(AUTOMATION)
  templateId      String?
  replyToCommentId String?  // comment → private reply trace

  createdAt       DateTime @default(now())
  deliveredAt     DateTime?
  readAt          DateTime?

  @@unique([orgId, platformMessageId])
}

```



```

    @@index([conversationId, createdAt])
  }

enum Direction { INBOUND OUTBOUND }
enum MsgType    { TEXT IMAGE VIDEO AUDIO FILE STICKER TEMPLATE STORY_REPLY COMMENT }
enum MsgStatus  { PENDING QUEUED SENT DELIVERED READ FAILED }
enum SentBy     { AUTOMATION AI HUMAN SYSTEM }

```

SCHEMA.PRISMA - AUTOMATION, TEMPLATES, OPS

```

model AutomationRule {
  id          String    @id @default(uuid())
  orgId       String
  channelId   String?    // null = all channels of that platform
  trigger     TriggerType // DM_RECEIVED | COMMENT_RECEIVED | STORY_REPLY | KEYWORD
  conditions  Json       // keyword list, regex, media id, business hours
  actions     Json       // reply, private_reply, tag, assign, ai_answer, template
  priority    Int        @default(0)
  isEnabled   Boolean    @default(true)
  runs        RuleRun[]
}

model RuleRun {
  id          String    @id @default(uuid())
  ruleId      String
  rule        AutomationRule @relation(fields: [ruleId], references: [id], onDelete: Cascade)
  messageId   String?
  outcome     String    // MATCHED | SKIPPED_WINDOW | FAILED | RATE_LIMITED
  detail      Json?
  createdAt   DateTime  @default(now())
  @@index([ruleId, createdAt])
}

model MessageTemplate { // WhatsApp only
  id          String    @id @default(uuid())
  orgId       String
  channelId   String
  metaTemplateId String?
  name        String
  language    String
  category    String    // MARKETING | UTILITY | AUTHENTICATION
  status      String    // PENDING | APPROVED | REJECTED | PAUSED | DISABLED
  rejectionReason String?
  components  Json
  @@unique([channelId, name, language])
}

```

```

model OptIn {
    // WhatsApp opt-in ledger – compliance proof
    id          String    @id @default(uuid())
    contactId   String
    contact     Contact    @relation(fields: [contactId], references: [id], onDelete: Cascade)
    source      String     // WEBSITE_FORM | CHECKOUT | IMPORT | INBOUND_MESSAGE
    evidence    Json       // ip, timestamp, page url, consent text shown
    optedInAt   DateTime   @default(now())
    optedOutAt  DateTime?

}

model PublishJob {
    // Reels / posts
    id          String    @id @default(uuid())
    orgId       String
    channelId   String
    mediaType   String     // REELS | IMAGE | CAROUSEL | STORY
    mediaUrl    String
    caption     String?
    scheduledAt DateTime?
    containerId String?     // Meta container id
    publishedId String?     // final media id
    status      String      @default("DRAFT") // DRAFT QUEUED UPLOADING PROCESSING PUBLISHED
    attempts    Int         @default(0)
    lastError   Json?
    @@index([orgId, status, scheduledAt])

}

model WebhookEvent {
    // raw log – replay + audit
    id          String    @id @default(uuid())
    object      String
    channelId   String?
    signatureOk Boolean
    payload     Json
    processedAt DateTime?
    error       Json?
    receivedAt  DateTime   @default(now())
    @@index([object, receivedAt])

}

model AuditLog {
    id          String    @id @default(uuid())
    orgId       String
    actorId     String?
    action      String     // CHANNEL_CONNECTED | CHANNEL_DISCONNECTED | TEMPLATE_SENT | ...
    target      String?
    meta        Json?
    createdAt   DateTime   @default(now())
    @@index([orgId, createdAt])

}

```

Messaging Policy Windows

Ye purely product constraint hai, lekin isko **schema aur send-path me encode karna padta hai** — warna aapka automation policy violation kar dega aur channel block ho jaayega.

SURFACE	WINDOW	WINDOW KE BAAD	CATCH
Messenger	24 hours from last user message	Message tags (ACCOUNT_UPDATE, POST_PURCHASE_UPDATE, CONFIRMED_EVENT_UPDATE) ya HUMAN_AGENT tag (7 din)	Tag ka misuse = enforcement
IG Direct	24 hours	HUMAN_AGENT tag — 7 din, sirf real human response ke liye	AI reply ko human agent tag mat lagao
IG comment → DM	7 din comment se	—	private_replies per comment sirf ek baar
Story reply	24 hours	—	Story 24h me expire, context save kar lo
WhatsApp	24-hour customer service window	Sirf approved template	Opt-in mandatory; per-message pricing

IMPLEMENTATION

- `Conversation.lastInboundAt + windowExpiresAt` — send karne se pehle **hamesha** check, DB me, na ki error aane par pata chale
- Window band hai to send path automatically decide kare: WA par template suggest karo, IG/FB par queue karke agle inbound par bhejo ya drop karo — **silent failure kabhi mat karo**, agent ko UI me dikhao
- WhatsApp template lifecycle: create → Meta approval → `message_template_status_update` webhook se status sync → rejected par reason UI me. Approved template me hi variables fill karke bhejo.
- Opt-in ledger (`OptIn` model) — marketing template bhejne se pehle proof hona chahiye.

Ye legal exposure hai, feature nahi.

- Har outbound par `AuditLog` — kis rule ne, kis window state me, kaunsa template bheja

11 Outbound Pipeline & Rate Limits

Har outbound call queue se jaayegi, kabhi seedha request handler se nahi. Bina queue ke burst me 613 errors aur WhatsApp number ki quality rating gir jaati hai.

- **Per-channel rate limiter** (Redis token bucket). FB/IG response headers `X-Business-Use-Case-Usage` / `X-App-Usage` parse karo — `call_count`, `total_cputime`, `total_time` me se koi 75% cross kare to throttle, 90% par pause.
- **Instagram content publishing**: 50 posts / 24 hours. `GET /{ig-id}/content_publishing_limit` se check karo publish se pehle.
- **WhatsApp messaging tiers**: 1K → 10K → 100K → unlimited unique customers/24h. Tier aur throughput metadata me store karo aur campaign scheduler usko respect kare.
- **Quality rating**: `phone_number_quality_update` webhook par YELLOW/RED aaye to marketing sends auto-pause + org ko alert.
- **Retry policy**: exponential backoff with jitter; 5xx aur rate-limit codes retryable; 100/131026 jaise permanent errors non-retryable → DLQ.
- **Ordering**: ek conversation ke messages FIFO rahein — per-conversation queue key use karo, warna replies out of order dikhengi.
- **Kill switch**: per-org aur global "stop all outbound" flag. Jis din automation loop me phase jaoge, ye bachaayega.

12 Reels Publishing Pipeline

MISSING FROM ORIGINAL RESEARCH

App ka naam `ReelToRealAuto` hai, lekin publish flow plan me tha hi nahi. Ye **asynchronous**, **multi-step** aur **failure-prone** hai — proper job state machine ke bina nahi chalega.

INSTAGRAM REELS — TEEN STEP

- 01 **Container create**: `POST /{ig-user-id}/media` with `media_type=REELS`, `video_url` (publicly reachable), `caption`, optional `cover_url`, `share_to_feed`, `thumb_offset`.
- 02 **Status poll**: `GET /{container-id}?fields=status_code,status` → `IN_PROGRESS` → `FINISHED` | `ERROR`. Ye 30 second se kuch minute le sakta hai. Poll interval 5s se shuru,

exponential, max ~10 min, uske baad fail.

03 **Publish:** POST `/ig-user-id/media_publish` with `creation_id`. Response me final media id → `PublishJob.publishedId`.

ZAROORI DETAILS

- **Video hosting:** Meta aapke URL se video *pull* karta hai — S3/R2 + CDN par public, stable URL chahiye. Signed URL ki expiry itni lambi rakho ki processing khatam ho jaaye. Bade files ke liye resumable upload API consider karo.
- **Format constraints:** MP4/MOV, H.264 + AAC, supported aspect ratio aur duration limits. Upload par server-side validate karo (ffprobe) — Meta ka reject message user ke liye useless hota hai.
- **Facebook Reels alag endpoint hai** — cross-post ko ek hi call maan kar mat chalo; do independent jobs banao taaki ek fail hone par doosra publish ho jaaye.
- **Scheduling:** org timezone me schedule, UTC me store. Scheduler job ko idempotent rakho (double publish sabse bura outcome hai).
- **Failure UX:** job fail ho to user ko exact reason + retry button. Draft aur media preserve rakho.
- **Comment automation hook:** publish ke baad us media id ko automation rules se link karo ("is reel par 'price' comment aaye to DM bhejo") — yahi app ka core loop hai.

13 Automation & AI Engine

Rules aur AI ko alag layers rakho: **deterministic rules pehle**, AI fallback baad me. Isse behaviour predictable rehta hai aur cost control me rehti hai.

EXECUTION PIPELINE

```
inbound event → resolve channel/org → contact + conversation upsert
→ window check → rule matcher (priority order)
  ├── exact/keyword match → scripted reply (no AI cost)
  ├── comment trigger → public reply + private_reply (7d, once)
  └── no match + aiEnabled → AI answer from knowledge base
→ outbound queue → audit log
```

AI LAYER

- **Knowledge base per org** — products, pricing, FAQs, policies. Chunk + embed, retrieval ke saath answer. Hallucinated pricing sabse bada business risk hai, isliye "sirf provided

context se answer do, warna handoff" system prompt me hard rule ho.

- **Confidence + handoff** — AI unsure ho, ya user "human se baat karao" bole, ya negative sentiment ho → `aiEnabled=false` , conversation assign to agent, notification.
- **Guardrails** — per-conversation AI reply cap (loop protection), per-org daily token budget, aur echo detection (apne hi outbound message par trigger na ho).
- **Cost metering** — har AI reply par tokens + cost record karo (§17). Yahi aapka variable cost hai.
- **Business hours** — off-hours me AI, on-hours me human routing — org setting.
- **Test mode** — rule ko live karne se pehle sandbox conversation par simulate karne ka UI. Clients isko sabse zyada maangenge.

14

Security & Secrets

- **Envelope encryption** — AES-256-GCM data key, master key KMS me (AWS KMS / GCP KMS / Vault). `keyVersion` column rotation enable karta hai bina downtime ke.
- **Tokens kabhi log mat karo** — logger me redaction filter (`access_token` , `code` , `pin` , `signature`), aur error objects me bhi (axios error me full request URL aa jaata hai, jisme token hota hai — ye classic leak hai).
- **App secret** sirf server side; `appsecret_proof` enable karo outgoing Graph calls par.
- **Webhook verify token** aur app secret alag-alag environments me alag.
- **Tenant isolation test** — CI me ek test jo dusre org ke resource ko fetch karne par 404 expect kare. Har naye endpoint ke liye.
- **PII & retention** — message bodies me customer PII hoti hai. Retention policy fix karo (e.g. 12 months), purge job likho, aur org-level export + delete API do.
- **Compliance** — Meta Platform Terms ke data-use limits, India ka DPDP Act (consent, purpose limitation, deletion on request), aur clients ke liye ek DPA template. Data deletion callback (§6) isi ka technical hissa hai.
- **Media** — WhatsApp/IG media ko apne storage me copy karte ho to signed URLs + expiry, public bucket kabhi nahi.

15

Observability

- **WebhookEvent raw log** — already schema me. Replay command isi par bane.
- **Outbound call log** — endpoint, channel, latency, response code, Meta error code, rate-limit

headers. Isse pata chalta hai ki throttle kahan lag raha hai.

- **Per-channel health dashboard** — status, last inbound event, last successful send, token expiry, WA quality rating. Support ke liye ye pehli screen hai.
- **Alerts:** webhook signature failures spike, DLQ depth > 0, koi channel 24h se NEEDS_REAUTH , error code 368 kahin bhi, publish job failure rate, AI cost spike.
- **Correlation id** — webhook event id se lekar outbound message tak ek hi trace id, taaki "is customer ka reply kyun nahi gaya" 2 minute me answer ho.
- **Uptime monitor** webhook endpoint par — agar Meta ko 5xx mile to wo subscription disable kar dega, aur aapko pata bhi nahi chalega.

16

Environments & Config

- **Teen Meta apps** — dev, staging, production. Har ek ka apna app id/secret, webhook URL, config ids. Production app ko kabhi dev testing ke liye mat use karo.
- **Local development** — ngrok/cloudflared tunnel, dev app ka webhook usi par. Test users aur test Page/IG/WABA banao (WhatsApp test number free messages deta hai).
- **Graph version as config** — META_GRAPH_VERSION=v24.0 . Channel-level override column bhi hai taaki phased migration ho sake.
- **Version upgrade cadence** — har naye version par ek ticket: changelog padho, breaking changes list karo, staging me test, fir rollout. Sunset date se 3 mahine pehle deadline.
- **Config ids** (Login for Business, WhatsApp ESU) env me, hardcoded nahi — alag environments me alag hote hain.
- **Feature flags** per-org — naye channel ya AI feature ko pehle apne test orgs par rollout karo.

17

Billing & Metering

SaaS hai, aur iski **variable cost real hai** — WhatsApp per-message charge karta hai aur AI replies tokens kha rahi hain. Ye Day 1 se meter karo, warna pricing guess-work ban jaayegi.

- **Plan limits:** channels count, monthly conversations, AI replies, scheduled posts, team seats.
- **Usage metering:** UsageRecord(orgId, metric, quantity, period) — metrics: ai_reply , wa_template_sent , publish_job , active_conversation .
- **WhatsApp costs:** Meta per message charge karta hai (cost-per-1k-messages). Do models

- **WhatsApp cost:** Meta per-message charge karta hai (category ke nisaab se). Do models —
(a) client apna payment method WABA par lagaye (aap sirf software fee lo, zero float), ya
(b) aap Tech Provider ke roop me credit line share karo aur markup ke saath bill karo
(better margin, lekin aapka cash risk aur reconciliation ka kaam). Ye \$19 ka open decision hai.
- **Enforcement:** limit cross hone par soft block (warning + grace) fir hard block. Inbound receiving kabhi block mat karo — sirf outbound/AI. Data loss se bura kuch nahi.
- **Subscription:** Razorpay/Stripe, invoices, proration, dunning emails, failed payment par grace period.

18

Build Roadmap

Order deliberately chuna gaya hai: pehle wo cheezein jo baad me retrofit karna mehenga hai (auth grant, webhook spine, domain model), aur App Review parallel me kyunki wo calendar time hai, engineering time nahi.

Phase 0
parallel

Meta paperwork — abhi shuru

Business Verification, privacy/ToS pages live, data-deletion + deauthorize endpoints, dev/staging/prod apps, test assets. Ye blocking hai aur calendar time leta hai — code ke saath chalao.

Phase 1

Foundation

User/Org/OrgMember + auth, MetaConnection + Channel models, envelope encryption + KMS, Prisma tenant middleware, config/env setup, base admin UI.

Phase 2

Webhook spine

GET verify, signature verification, raw event store, queue + worker + DLQ, replay command, routing skeleton for teeno objects. Abhi koi business logic nahi — sirf reliable plumbing.

Phase 3

Instagram vertical slice

Connect flow + subscribed_apps, Contact/Conversation/Message, DM + comment inbound, private replies, 24h window enforcement, keyword rules, basic inbox UI. Yahan poora domain model settle ho jaayega.

Phase 4

Facebook parity

Page channel, Messenger in/out, feed comments. Zyadatar code Phase 3 se reuse hoga — agar nahi ho raha to abstraction galat hai, wahin theek karo.

Phase 5

WhatsApp

Embedded Signup, phone register + PIN, WABA webhooks, template sync + status webhook, opt-in ledger, 24h window + template send path, quality monitoring.

Phase 6

Publishing

Media upload + validation, IG Reels container/poll/publish state machine, FB Reels, scheduler, failure UX, publish→automation linkage.

Phase 7

AI engine

Knowledge base ingestion, retrieval, guardrails, handoff, test mode, cost metering.

Phase 8

Commercial

Plans, usage metering + enforcement, subscriptions, invoices, WhatsApp cost pass-through.

Phase 9

Hardening & launch

Health cron, alerting, tenant isolation tests, load test on webhook path, App Review submission with demo org, staged rollout.

19

Open Decisions

Ye faisle architecture ko badalte hain — inhe code likhne se pehle close karo.

#	DECISION	IMPACT	RECOMMENDATION
1	Ek Page/IG do orgs me connect ho sakta hai?	Webhook routing, uniqueness constraint	Nahi — global unique + explicit takeover flow
2	WhatsApp billing: client ka payment method vs credit line sharing	Margin, cash risk, onboarding friction	V1 me client ka apna method; credit line tab jab volume aaye
3	Standalone Instagram Login support karein?	Do token strategies, do endpoint sets, support load	V1 me sirf Page-linked; standalone Phase 2
4	AI model + per-reply cost ceiling	Unit economics, pricing	Cheap model default + escalation, per-org budget cap
5	Message retention period	Storage cost, DPDP, client trust	12 months default, plan-based, purge job Day 1

6	Hosting region	Latency, data residency	India region agar clients India me hain
7	Self-serve WABA onboarding vs assisted	Support cost, conversion	Self-serve Embedded Signup + assisted fallback
8	Agency/sub-account hierarchy	Schema (Org parent-child)	V1 me nahi, par Organization.parentId ke liye jagah chhodo

